

## 54. Spiral Matrix Medium

Acceptance: 55.9% | [LeetCode](#)

Given an  $m \times n$  matrix, return *all elements of the matrix in spiral order*.

**Example 1:**

```
**Input:** matrix = [[1,2,3],[4,5,6],[7,8,9]]
**Output:** [1,2,3,6,9,8,7,4,5]
```

**Example 2:**

```
**Input:** matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
**Output:** [1,2,3,4,8,12,11,10,9,5,6,7]
```

**Constraints:**

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $-100 \leq \text{matrix}[i][j] \leq 100$

Claude Opus Solution

## Spiral Matrix — Complete Solution

### 1. Intuition

The key insight is that traversing a matrix in spiral order means peeling off the **outer layer** of the matrix one boundary at a time, like peeling an onion. Each layer consists of 4 directional moves:

```
→ go RIGHT along the top row
↓ go DOWN along the right column
← go LEFT along the bottom row
↑ go UP along the left column
```

After completing one full loop, we **shrink the boundaries inward** and repeat until all elements are visited.

The natural way to track this is with **4 pointers** that define the current "unvisited box": - `top`, `bottom` (row bounds) - `left`, `right` (column bounds)

### 2. Approach (Pseudocode)

1. Initialize boundaries: `top=0`, `bottom=m-1`, `left=0`, `right=n-1`
2. While `top <= bottom` AND `left <= right`:
  - a. Traverse top row, `left → right` ; then `top++`
  - b. Traverse right col, `top → bottom` ; then `right--`
  - c. IF `top <= bottom`: traverse bottom row `right → left` ; then `bottom--`
  - d. IF `left <= right`: traverse left col `bottom → top` ; then `left++`
3. Return the collected list

**Note on the reference solution:** It uses `top < bottom` and `left < right` which fails for single-row/single-column matrices inside the loop, then handles the remainder with a final nested loop. My approach below uses `<=` with **guard checks**, which is cleaner and handles all cases inside the loop.

### 3. Python Solution

```
from typing import List

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        result = []

        # Define the four boundaries of the "current box" we are processing.
        top, bottom = 0, len(matrix) - 1          # row boundaries
        left, right = 0, len(matrix[0]) - 1        # column boundaries

        # Continue while the box is still valid (has at least one cell).
        while top <= bottom and left <= right:
```

```

# --- 1. Traverse the TOP row from left to right ---
for col in range(left, right + 1):
    result.append(matrix[top][col])
top += 1 # The top row is done; move boundary down.

# --- 2. Traverse the RIGHT column from top to bottom ---
for row in range(top, bottom + 1):
    result.append(matrix[row][right])
right -= 1 # The right column is done; move boundary left.

# --- 3. Traverse the BOTTOM row from right to left ---
# Guard: only if there's still a row left to process.
# (Prevents re-reading a row in single-row matrices.)
if top <= bottom:
    for col in range(right, left - 1, -1):
        result.append(matrix[bottom][col])
    bottom -= 1 # The bottom row is done; move boundary up.

# --- 4. Traverse the LEFT column from bottom to top ---
# Guard: only if there's still a column left to process.
if left <= right:
    for row in range(bottom, top - 1, -1):
        result.append(matrix[row][left])
    left += 1 # The left column is done; move boundary right.

return result

```

### Walkthrough on Example 1: `[[1,2,3],[4,5,6],[7,8,9]]`

Box: top=0,bottom=2,left=0,right=2  
 Top row L→R: 1, 2, 3 → top=1  
 Right col T→B: 6, 9 → right=1  
 Bottom row R→L: 8, 7 → bottom=1  
 Left col B→T: 4 → left=1

Box: top=1,bottom=1,left=1,right=1  
 Top row L→R: 5 → top=2  
 Right col T→B: (empty) → right=0  
 Guard top(2)<=bottom(1)? NO → skip  
 Guard left(1)<=right(0)? NO → skip

Loop ends (top > bottom). Result: `[1,2,3,6,9,8,7,4,5]` ✓

## 4. Complexity Analysis

| Metric | Complexity      | Explanation                                                                                                                      |
|--------|-----------------|----------------------------------------------------------------------------------------------------------------------------------|
| Time   | $O(m \times n)$ | Every element is visited <b>exactly once</b> . Total elements = $m \times n$ .                                                   |
| Space  | $O(1)$ extra    | We only use a constant number of pointers. (The output list is $O(m \times n)$ but that's required output, not auxiliary space.) |

## 5. Alternative Approaches

**A. Direction Vectors + Visited Set** Simulate movement using direction deltas `[(0,1),(1,0),(0,-1),(-1,0)]`. Mark cells as visited; when you'd go out of bounds or onto a visited cell, rotate direction. - *Pros*: Conceptually simple, no boundary bookkeeping. - *Cons*: Uses  $O(m \times n)$  extra space for the visited set.

```

def spiralOrder(matrix):
    m, n = len(matrix), len(matrix[0])
    seen = [[False] * n for _ in range(m)]
    dr, dc = [0, 1, 0, -1], [1, 0, -1, 0] # right, down, left, up
    r = c = d = 0
    result = []
    for _ in range(m * n):
        result.append(matrix[r][c])
        seen[r][c] = True
        nr, nc = r + dr[d], c + dc[d]
        # Turn if next cell is out of bounds or already visited
        if not (0 <= nr < m and 0 <= nc < n and not seen[nr][nc]):
            d = (d + 1) % 4
            r, c = nr, nc
    return result

```

```
        nr, nc = r + dr[d], c + dc[d]
    r, c = nr, nc
    return result
```

**B. Recursive Layer Peeling** Recursively process the outer layer, then recurse on the inner submatrix. Functionally identical to the iterative boundary approach but uses call-stack space.

**C. Transpose-and-Reverse (functional trick)** Repeatedly take the first row, then rotate the remaining matrix counterclockwise (`matrix = list(zip(*matrix))[:-1]`). Elegant in Python but creates new matrices each step → less efficient.

---

### Recommended Solution

The **boundary-pointer approach** (Section 3) is the best balance of clarity and efficiency: `O(1)` extra space, single pass, and the guard checks make it robust for edge cases like single rows/columns.

---

## Codex Solution

### 1. Intuition

A spiral traversal is just repeatedly walking around the **outer border** of the remaining matrix:

1. Move **left** → **right** across the top row.
2. Move **top** → **bottom** down the right column.
3. Move **right** → **left** across the bottom row.
4. Move **bottom** → **top** up the left column.

After completing one outer layer, we "shrink" the boundaries inward and repeat.

We can track the current unvisited rectangle using four boundaries:

```
top, bottom, left, right
```

Initially:

```
top = 0
bottom = m - 1
left = 0
right = n - 1
```

After traversing one layer:

```
top += 1
bottom -= 1
left += 1
right -= 1
```

The key is to avoid duplicates when there is only one row or one column left, so we check boundaries before traversing the bottom row and left column.

---

## 2. Approach

### Pseudocode

```
function spiralOrder(matrix):
    result = []

    top = 0
    bottom = number of rows - 1
    left = 0
    right = number of columns - 1

    while top <= bottom and left <= right:

        # 1. Traverse top row from left to right
        for col from left to right:
            add matrix[top][col] to result
        top += 1
```

```

# 2. Traverse right column from top to bottom
for row from top to bottom:
    add matrix[row][right] to result
    right -= 1

# 3. Traverse bottom row from right to left
if top <= bottom:
    for col from right down to left:
        add matrix[bottom][col] to result
    bottom -= 1

# 4. Traverse left column from bottom to top
if left <= right:
    for row from bottom down to top:
        add matrix[row][left] to result
    left += 1

return result

```

### 3. Python Solution

```

from typing import List

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        # Result list to store elements in spiral order
        result = []

        # Define the initial boundaries of the matrix
        top = 0
        bottom = len(matrix) - 1
        left = 0
        right = len(matrix[0]) - 1

        # Continue while there is still a valid rectangle to traverse
        while top <= bottom and left <= right:

            # 1. Traverse the top row from left to right
            for col in range(left, right + 1):
                result.append(matrix[top][col])

            # The top row has now been fully visited
            top += 1

            # 2. Traverse the right column from top to bottom
            for row in range(top, bottom + 1):
                result.append(matrix[row][right])

            # The right column has now been fully visited
            right -= 1

            # 3. Traverse the bottom row from right to left
            # We only do this if there is still an unvisited row remaining.
            #
            # This check prevents duplicate traversal when the matrix has
            # only one row left.
            if top <= bottom:
                for col in range(right, left - 1, -1):
                    result.append(matrix[bottom][col])

            # The bottom row has now been fully visited
            bottom -= 1

            # 4. Traverse the left column from bottom to top
            # We only do this if there is still an unvisited column remaining.
            #
            # This check prevents duplicate traversal when the matrix has
            # only one column left.
            if left <= right:
                for row in range(bottom, top - 1, -1):

```

```
        result.append(matrix[row][left])

        # The left column has now been fully visited
        left += 1

    return result
```

---

## 4. Complexity Analysis

Let:

```
m = number of rows
n = number of columns
```

### Time Complexity

```
O(m * n)
```

Each element in the matrix is visited exactly once.

### Space Complexity

```
O(1) auxiliary space
```

We only use a few boundary variables.

However, the returned output list stores all  $m * n$  elements, so if we count the output space:

```
O(m * n)
```

---

## 5. Alternative Approaches

### 1. Direction Simulation

Another common approach is to simulate movement using direction vectors:

```
directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
```

You move in one direction until you hit a boundary or a visited cell, then turn right.

This works well, but requires either:

- a `visited` set/matrix, or
- careful boundary checks.

Its complexity is also:

```
Time: O(m * n)
Space: O(m * n) if using visited
```

### 2. Recursive Layer Traversal

You can recursively process one outer layer, then call the same function on the inner submatrix.

This matches the spiral idea nicely, but recursion adds overhead and can be less clean in Python.

### Recommended Pattern

For this problem, the **four-boundary shrinking window** approach is the cleanest and most space-efficient.